# Material didático - Estruturas de Decisão

Site: Ambiente Virtual de Aprendizagem - UFR
Course: ALGORITMOS E PROGRAMAÇÃO
Book: Material didático - Estruturas de Decisão

Printed by: Nabor do Nascimento da Silva
Date: Sunday, 5 April 2026, 4:50 PM

# Description

# Table of contents

**1. Estruturas de Decisão**

**2. Lógica de Boole**
2.1. Operadores lógicos
2.2. Testes lógicos
2.3. Tabelas verdade
2.4. Testes aninhados

**3. Categorias de problemas**

**4. Representações da Estrutura de Decisão**
4.1. Fluxograma
4.2. Portugol
4.3. Python

**5. Indentação**

**6. Como testar Estruturas de Decisão**
6.1. Testes de Mesa

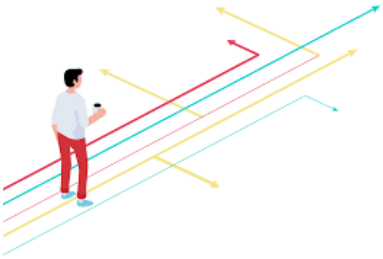
**7. Estrutura Composta em Python**

**8. Estrutura Composta Encadeada em Python**

**9. Encadeamentos complexos**

**10. Estrutura if-elif-else**
10.1. Exemplo: calculadora

# 1. Estruturas de Decisão



### Estruturas de Decisão

Estruturas de Decisão, também conhecidas como Estruturas Condicionais, ou ainda Estruturas de Seleção, são um elemento fundamental na programação de computadores.

Diversas situações na vida nos obrigam e realizar testes e verificar se o resultado é falso ou verdadeiro. Este tipo de teste é conhecido por TESTE LÓGICO.

Por exemplo, se tivermos sede (e se o resultado for verdadeiro), precisamos beber água.

Da mesma forma, diversos problemas computacionais demandam esse tipo de teste. Vejam os exemplos:

- Verificar se o divisor informado é zero e não deixar realizar a divisão;
- Aplicar uma fórmula se o sexo informado for masculino e outra fórmula se o sexo informado for feminino;
- Aplicar um desconto se o valor do produto for acima de determinado valor;
- Verificar se um número é positivo, negativo ou zero;
- Verificar se um número é maior, menor ou igual a outro.

Felizmente, os computadores conseguem realizar testes e verificar seus resultados. Esses testes são chamados de testes lógicos ou booleanos.

O nome booleano é uma referência ao trabalho de George Boole. Tire um tempinho para ler o trabalho desse importante personagem na história da computação.

- Link: https://pt.wikipedia.org/wiki/George_Boole

# 2. Lógica de Boole

O termo "álgebra booliana" é uma homenagem a George Boole, um matemático inglês autodidata.

Boole introduziu o sistema algébrico, inicialmente, em um pequeno panfleto, o The Mathematical Analysis of Logic, publicado em 1847, em resposta a uma controvérsia em curso entre Augustus De Morgan e William Hamilton, e mais tarde como um livro mais substancial, The Laws of Thought, publicado em 1854.

O matemático britânico **George Boole (1815-1864)** que inventou um sistema de álgebra que é chave para a programação de hoje - conhecido como sistema booleano em sua homenagem.

# 2.1. Operadores lógicos

## Operadores lógicos

Os operadores lógicos, também conhecidos como operadores de comparação, são utilizados na lógica e na programação para comparar valores e determinar relações de igualdade, desigualdade e outros tipos de comparações entre variáveis ou expressões.

**Exemplo**:
- Para a determinação das raizes reais de uma equação de segundo grau, primeiro é necessário verificar se o delta é maior ou igual a zero.
- A sentença de comparação ou teste é escrita com um operador lógico relacional, veja:
- delta >= 0

Os operadores lógicos são amplamente utilizados em linguagens de programação e em expressões lógicas para controlar o fluxo do programa ou para criar condições que afetam o comportamento do código.

Os principais operadores lógicos relacionais e suas representações na linguagem de programação Python:

| Operador | Significado |
|---|---|
| == | Igual a |
| ! = | Diferente de |
| > | Maior que |
| < | Menor que |
| >= | Maior ou igual a |
| <= | Menor ou igual a |

**Valor Booleano**

As operações lógicas retornam um valor booleano (verdadeiro ou falso) que indica se a condição especificada na comparação é verdadeira ou falsa.

Por exemplo,

a expressão `x > y` retornará:

- verdadeiro se o valor de "x" for maior que o valor de "y";
- caso contrário, retornará falso.

**Uso na computação**

Os operadores lógicos relacionais são fundamentais para a tomada de decisões em programas, pois permitem que o código execute diferentes ações com base nas condições que são verificadas.

Essas condições são comumente usadas em declarações de controle de fluxo, complementares aos comandos de estruturas de decisão "if" (se) e também em estruturas de repetição "while".

Além disso, eles são cruciais para a construção de expressões lógicas mais complexas ao serem combinados com operadores lógicos AND, OR, NOT.

**Operadores lógicos conectivos**

| Operador | Definição | Execução |
|---|---|---|
| and | E | Retorna True se ambas as afirmações forem verdadeiras |
| or | OU | Retorna True se uma das afirmações for verdadeira |
| not | NÃO | Retorna Falso se o resultado for verdadeiro |

# 2.2. Testes lógicos

**Testes lógicos**

A base do funcionamento dos testes lógicos são pressuposições de que um valor (ou resultado de uma operação) pode ser verdadeiro ou falso.

O computador fará o teste e retornará VERDADEIRO ou FALSO.

## Tabela: exemplos de testes e resultados em Python

| **Teste** | **Sentença de teste** | **resultado** |
|---|---|---|
| print (1 == 1) | um é igual a 1 | true |
| print (1 == 0) | um é igual a zero | false |
| print (1 != 0) | um é diferente de zero | true |
| print (1 > 0) | um é maior que zero | true |
| print (1 < 0) | um é menor que um | false |
| print (1>=0) | um é maior ou igual a 0 | true |
| print (-1<=0) | menos um é menor ou igual a zero | true |

# 2.3. Tabelas verdade

Uma tabela verdade é uma representação, na forma de uma tabela, de todas as combinações possíveis para um conjunto de valores(ou variáveis) em uma expressão lógica.

Vamos considerar um exemplo simples com duas sentenças:

- 1 = 1
- 1 >= 0

A lógica de Boole propõe que podemos realizar operações com as duas sentenças. As comparações mais comuns são realizadas com conectivos E e OU.

- comparação, usando o conectivo E
- comparação, usando o conectivo OU

O conectivo E estabelece que as duas sentenças comparadas precisam ser verdadeiras para que o resultado da comparação seja também verdadeiro.

**Exemplos**:

- (1+1 = 2) E (1>=0) o resultado será verdadeiro.
- (1+1 = 2) E (1>=3) o resultado será falso.
- (1+1 = 3) E (1>=0) o resultado será falso.
- (1+1 = 3) E (1>=3) o resultado será falso.

Se considerarmos a primeira sentença como A e a segunda como B, as possíveis combinações seriam:

## Tabela Verdade para o conectivo E

| Sentença A | Conctivo E | Sentença B | Resultado lógico |
|---|---|---|---|
| V | E | V | V |
| V | E | F | F |
| F | E | V | F |
| F | E | F | F |

O conectivo OU estabelece que pelo menos uma das duas sentenças comparadas precisam ser verdadeiras para que o resultado da comparação seja verdadeiro.

**Exemplos**:

- (1+1 = 2) OU (1>=0) o resultado será verdadeiro.
- (1+1 = 2) OU (1>=3) o resultado será verdadeiro.
- (1+1 = 3) OU (1>=0) o resultado será verdadeiro.
- (1+1 = 3) OU (1>=3) o resultado será falso.

## Tabela Verdade para o conectivo OU

| Sentença A | Conectivo OU | Sentença B | Resultado lógico |
|---|---|---|---|
| V | OU | V | V |
| V | OU | F | V |

| Sentença A | Conectivo OU | Sentença B | Resultado lógico |
|---|---|---|---|
| F | OU | V | V |
| F | OU | F | F |

# 2.4. Testes aninhados

**Testes aninhados**

Da mesma forma que utilizamos expressões matemáticas com aninhamentos, ou seja, uma operação interna a outra, ou dentro de outra.

Com o uso de parênteses, podemos utilizar expressões lógicas com testes aninhados, ou seja, testes dentro de testes.

Por exemplo:

- ( ( (A=1 E B=0) OU (C>0 OU D<=1) ) OU (A!=0 E C>0) )

# 3. Categorias de problemas

Algumas das categorias de problemas mais comuns envolvendo a estrutura de decisão incluem:

1. **Decisões Simples:** Problemas que exigem uma única decisão com base em uma condição simples, como exibir uma mensagem ou executar uma ação específica se uma variável for igual a um valor específico.
2. **Decisões Múltiplas (Cascata ou Encadeadas):** Problemas que envolvem decisões múltiplas em que várias condições são testadas em sequência, com diferentes ações executadas dependendo do resultado de cada teste.
3. **Decisões com Operadores Lógicos:** Problemas que requerem o uso de operadores lógicos (AND, OR, NOT) para combinar várias condições em uma única decisão.
4. **Decisões Aninhadas:** Problemas em que uma decisão é feita dentro de outra decisão, formando uma estrutura aninhada de decisões.
5. **Decisões com Intervalos:** Problemas que envolvem a verificação se um valor está dentro de um intervalo específico e, com base nisso, tomar uma decisão.
6. **Decisões com Strings:** Problemas que envolvem o uso de decisões com base em comparações de strings, como verificar se duas strings são iguais ou diferentes.
7. **Decisões com Números Aleatórios:** Exercícios que requerem a geração de números aleatórios e tomada de decisões com base nesses números.
8. **Decisões com Entrada de Usuário:** Problemas que solicitam ao usuário que insira um valor e, com base nessa entrada, tomam decisões específicas.
9. **Decisões com Condições Compostas:** Problemas que combinam várias condições usando parênteses, para criar decisões mais complexas.
10. **Decisões com Variáveis Booleanas:** Problemas que envolvem o uso de variáveis booleanas para armazenar o resultado de uma decisão e utilizá-las em outras partes do programa.

# 4. Representações da Estrutura de Decisão

**Estrutura de Decisão Simples**

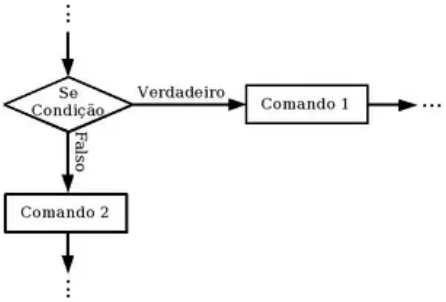
Vejamos a seguir as representações em

- Fluxograma
- Portugol
- Python

# 4.1. Fluxograma

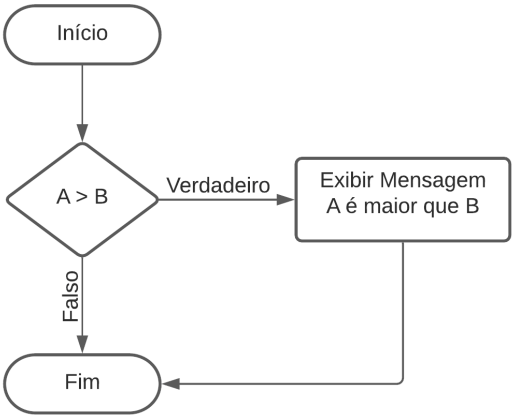
Veja a representação da Estrutura de Decisão Simples em fluxograma:

Figura - Fluxograma da Estrutura de Decisão Simples




Exemplo:

Elabore um fluxograma para um algorimto que leia A e B, que serão valores inteiros digitados pelo teclado. Determine e mostre somente se A é maior que B.



## 4.2. Portugol

A Estrutura de Decisão Simples em Portugol/VISUALG vai ter o seguinte formato:

**SE** (teste resultar em verdadeiro) **entao**
    <execução de um comando ou de um bloco de comandos>
**fimse**

Exemplo:

Elabore um algoritmo que leia A e B, que serão valores inteiros digitados pelo teclado. Determine e mostre somente se A é maior que B. Nesse caso mostrar a mensagem: A é maior que B

```
escreva ("Digite o valor de A")
leia (A)
escreva ("Digite o valor de B")
leia (B)
se (A > B) entao
  escreva ("A é maior que B")
fimse
```

# 4.3. Python

Em Python, a estrutura de decisão simples é implementada utilizando a instrução `if`, seguida por uma condição que é avaliada como verdadeira ou falsa.

Se a condição for verdadeira, o bloco de código indentado sob o `if` será executado. Caso contrário, o programa pode seguir para a próxima instrução fora do bloco `if`.

A sintaxe geral da estrutura de decisão simples em Python é a seguinte:

```
if (testeLogico for verdadeiro):
    # Bloco de código a ser executado se a condição for verdadeira
```

Exemplo:

Elabore um algorimto que leia A e B, que serão valores inteiros digitados pelo teclado. Determine e mostre somente se A é maior que B. Nesse caso mostrar a mensagem: A é maior que B

```
# Programa para verificar se A é maior que B

# Leitura dos valores A e B do teclado
A = int(input("Digite o valor de A: "))
B = int(input("Digite o valor de B: "))

# Verificação da condição e exibição da mensagem, se aplicável
if (A > B):
   print("A é maior que B.")
```

# 5. Indentação

Você percebeu que logo após o teste, antes do comando print, veio um um pequeno deslocamento à direita?

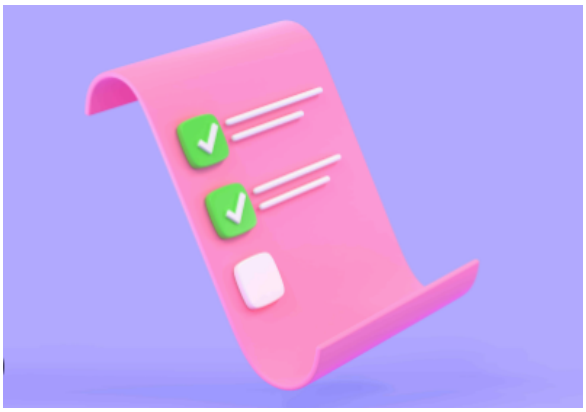
```
# Verificação da condição e exibição da mensagem, se aplicável
if A > B:
    print("A é maior que B.")
```

Trata-se de uma forma de indicar que tudo que está deslocado à direita deve ser executado uma vez que o teste for verdadeiro.

Isso ocorre obrigatoriamente em algumas linguagens de programação, como Python.

Caso não seja realizada a indentação, o comando print será obrigatoriamente executado.

# 6. Como testar Estruturas de Decisão



**Testes com Estrutura de Decisão**

Para elaborar testes para a estrutura de decisão simples, é importante considerar diferentes cenários possíveis para a condição que está sendo avaliada. O objetivo dos testes é garantir que a estrutura condicional funcione corretamente em todas as situações relevantes.

No caso do exemplo que demos anteriormente, onde o programa verifica se A é maior que B, podemos elaborar os seguintes testes:

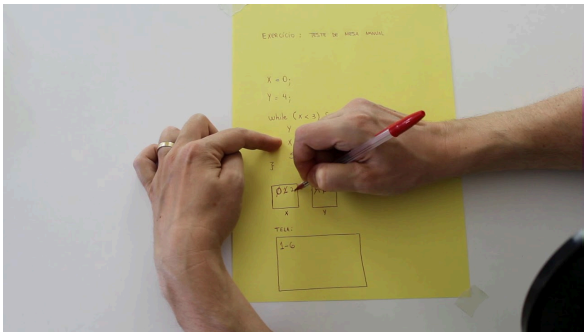
**Casos de teste possíveis**

| **Teste** | **Entradas** | **Saída esperada** |
|---|---|---|
| Teste com A maior que B: | Entrada: A = 10, B = 5 | Saída esperada: "A é maior que B." |
| Teste com B maior que A: | Entrada: A = 7, B = 12 | Saída esperada: (nenhuma mensagem, pois A não é maior que B) |
| Teste com A igual a B: | Entrada: A = 10, B = 10 | Saída esperada: (nenhuma mensagem, pois A não é maior que B) |
| Teste com valores negativos: | Entrada: A = -3, B = -8 | Saída esperada: "A é maior que B." |
| Teste com valores iguais a zero: | Entrada: A = 0, B = 0 | Saída esperada: (nenhuma mensagem, pois A não é maior que B) |

Esses são apenas alguns exemplos de testes que podem ser elaborados para verificar o funcionamento correto da estrutura de decisão simples.

É importante abranger diferentes faixas de valores e considerar casos extremos para garantir que o programa lide adequadamente com todas as situações possíveis.

# 6.1. Testes de Mesa



**Algoritmo:**

Elabore um algorimto que leia A e B, que serão valores inteiros digitados pelo teclado. Determine e mostre somente se A é maior que B. Nesse caso mostrar a mensagem: A é maior que B

```
# Programa para verificar se A é maior que B

# Leitura dos valores A e B do teclado
A = int(input("Digite o valor de A: "))
B = int(input("Digite o valor de B: "))

# Verificação da condição e exibição da mensagem, se aplicável
if (A > B):
   print("A é maior que B.")
```

**Caso 1: A maior que B: A = 2 e B = 1**

Resultado esperado: A é maior que B

| **No. da linha** | **Comando** | **Variáveis** | | **Tela** |
|---|---|---|---|---|
| | | A | B | |
| 1 | A = int(input("Digite o valor de A: ")) | 2 | | |
| 2 | B = int(input("Digite o valor de B: ")) | | 1 | |
| 3 | if (A > B): | | | |
| 4 | print("A é maior que B.") | | | A é maior que B.") |

**Caso 2: B maior que A: A = 1 e B = 2**

Resultado esperado: nenhum

| **No. da linha** | **Comando** | **Variáveis** | | **Tela** |
|---|---|---|---|---|
| | | A | B | |
| 1 | A = int(input("Digite o valor de A: ")) | 1 | | |
| 2 | B = int(input("Digite o valor de B: ")) | | 2 | |
| 3 | if (A > B): | | | |
| 4 | print("A é maior que B.") | | | |

**Caso 3: A = B: A = 1 e B = 1**

Resultado esperado: nenhum

| **No. da linha** | **Comando** | **Variáveis** | | **Tela** |
|---|---|---|---|---|
| | | A | B | |
| 1 | A = int(input("Digite o valor de A: ")) | 1 | | |
| 2 | B = int(input("Digite o valor de B: ")) | | 1 | |
| 3 | if (A > B): | | | |
| 4 | print("A é maior que B.") | | | |

# 7. Estrutura Composta em Python

A Estrutura de Decisão COMPOSTA em Python deve obedecer à seguinte lógica de construção:

| | |
|---|---|
| IF (  ):<br><br>COMANDO<br><br>ELSE:<br><br>COMANDO | O comando de teste IF deve conter os testes dentro de parënteses e finalizar com dois pontos ( : ).<br><br>Da mesma forma que a estrutura simples, podem haver diversos COMANDOS dentro da estrutura, que serão executados quando o teste retornar VERDADEIRO.<br><br>O que vai diferenciar é o ELSE.<br><br>ELSE é uma instrução de teste, que manda executar outro bloco de comandos caso o teste do IF seja FALSO.<br><br>A primeira instrução não endentada marca o fim do bloco. |

Exemplo:

| | |
|---|---|
| If ( x % 2 == 0):<br><br>print (x, ‘é par’)<br><br>else:<br><br>print (x, ‘é impar’) | Se o resto da divisão de x por 2 for 0, então sabemos que x é par, e o programa exibe a mensagem para esta condição.<br><br>Se a condição é falsa, o segundo grupo de instruções é executado. Desde que a condição deva ser verdadeira (True) ou falsa (False), precisamente uma das alternativas vai ser executada. |

As alternativas são chamadas ramos (branches), porque existem ramificações no fluxo de execução.

**Exemplo de algoritmo/programa com a estrutura composta**

```
# ========================================
'''
Seção de cabeçalho
Nome do algoritmo/programa: Lista estruturas de decisão -
Ex. 01
Função/Descrição: 1.    Escreva
um algoritmo que leia dois números inteiros.
Mostre na tela e diga se eles são iguais ou diferentes.
Autor: Waine Teixeira Júnior
Data: 31/05/2019
Número de controle de versão: 1.0
'''
# ========================================
'''
Seção de Declarações de variáveis

'''
# ========================================
# inicio do algoritmo/programa

#entrada de dados
print ('Digite o primeiro número')
n1 = int (input())
print ('Digite o segundo número')
n2 = int (input())

#processamento e saida

if (n1 == n2):
    print (n1, ' e ',n2,' são iguais')
else:
    print (n1, ' e ',n2,' são diferentes')

# ========================================
# fim do algoritmo/programa
```

Vejamos a seguir como construir estruturas compostas encadeadas ou aninhadas.

# 8. Estrutura Composta Encadeada em Python

Nas estruturas de decisão encadeadas em Python, **uma estrutura de condição é aninhada dentro de outra**.

Portanto, teremos as mesmas possibilidades de encadeamento vistas em Portugol para o VISUALG.

**a) Estruturas Condicionais Simples dentro de Estruturas Condicionais Simples**

```
IF (    ): #aqui o início a estrutura de decisão simples
     COMANDO
                COMANDO
          IF (    ): #aqui o início a estrutura de decisão simples encadeada (interna)
```

**b) Estruturas Condicionais Simples dentro de Estruturas Condicionais Compostas**

```
IF (    ): #aqui o início da estrutura de decisão composta
     COMANDO
ELSE:
     IF (    ):  #aqui o início a estrutura de decisão simples encadeada (interna)
          COMANDO
```

**c) Combinações entre Estruturas Condicionais Simples e Compostas**

```
IF (    ): #aqui o início da estrutura de decisão composta
     COMANDO
ELSE:
     IF (    ):  #aqui o início a estrutura de decisão composta
encadeada (interna)
          COMANDO
     ELSE
          IF (    ): #aqui o início da estrutura dde decisão composta
encadeada (interna)
                COMANDO
          ELSE:
                IF (    ):  #aqui o início a estrutura de decisão simples encadeada (interna)
                    COMANDO
```

---

A seguir vamos estudar a estrutura de decisão de Múltipla Escolha.

# 9. Encadeamentos complexos

Como já vimos até agora, embora todos os problemas que envolvem algum tipo de pré-condição podem ser solucionados com a estrutura simples. Mas há sutis diferenças nas soluções proporcionadas pela estrutura composta, de forma que pode deixar o código mais compreensível e inclusive otimizar o código, diminuindo a quantidade de testes, o que pode representar aumento de desempenho em situações nas quais o teste tenha que ser repetido muitas vezes.

A estrutura Composta pode aperfeiçoar o código pela possibilidade de diminuição do número de testes no código. Entretanto, depois que resolvemos as listas de exercícios, percebemos pela prática que, na grande maioria das vezes é muito mais fácil utilizar somente a estrutura composta.

Os problemas nos quais utilizamos a estrutura Composta nos parece fácil e lógico a sua utilização, são problemas relativamente simples, envolvendo um número pequeno de possibilidades.

Por exemplo, verificar se um número é positivo, negativo ou nulo requer 3 testes encadeados.

Entretanto, existem situações nas quais as estruturas condicionais se aninham e então o uso da estrutura SE/SENÃO fica altamente complexa, principalmente, existem situações nas quais:

- há muitas condições de execução
- e o grupo de comandos a ser executado dentro de cada condição é grande, dificultando a visualização e organização do código

Existe uma forma de solucionar esse problema. Vejamos a seguir.

# 10. Estrutura if-elif-else

A **sintaxe** do comando `if-elif-else` é a seguinte:

```
if condição:
    # bloco contendo comandos a serem executados
elif ( ):
     # bloco contendo comandos a serem executados
elif ( ):
     # bloco contendo comandos a serem executados
# outros possíveis blocos de elif, cada um com a sua condição
else:
     # bloco contendo comandos a serem executados
comando_apos_if
```

Ou seja, o `if-elif-else` pode ter um ou mais blocos com `elif`, cada um com a sua condição específica.

- Cada bloco (condição) é testado um de cada vez, até que uma condição seja satisfeita e apenas os comandos dentro desse bloco são executados.
- Quando nenhuma condição é satisfeita, os comandos dentro do `else` são executados. Assim, **não** é necessário que haja um `else` no final.

Vamos estudar um exemplo simples e comparar as soluções.

# 10.1. Exemplo: calculadora

Vamos estudar um exemplo mais complexo.

Elabore um algoritmo um menu de opções de operações matemáticas vai oferecer as seguintes possibilidades:

(1) soma,

(2) subtração,

(3) divisão,

(4) multiplicação,

(5)raiz quadrada,

(6) exponenciação.

(7) porcentagem, e por ai vai.

Para programar um menu de opções teriamos que utilizar a estrutura simples, uma vez que a composta deixaria o código bem complicado, com um grande encadeamento para poder otimizar o código.

Ao invés disso podemos utilizar a estrutura `if-elif-else`.

**Veja a seguir o código do algorimto da calculadora:**

```
print ("Digite a opção desejada")
print ("1 - para soma")
print ("2 - para subtração")
print ("3 - para multiplicação")
print ("4 - para divisão")
op = int(input())
if (op==1):
    print ("Você escolheu a soma.Digite o primeiro valor")
    n1 = int(input())
    print ("Digite o segundo valor a ser somado")
    n2 = int (input())
    soma = n1+n2
    print ("o valor da soma de ", n1, " por  ", n2, "  é ", soma)
elif (op==2):
    print ("Você escolheu a subtração. Digite o primeiro valor")
    n1 = int(input ())
    print ("Digite o segundo valor a ser subtraído")
    n2 = int(input ())
    subtracao = n1-n2
    print ("o valor da subtração de ", n1, " menos  ", n2, "  é ", subtracao)
elif (op==3):
    print ("Você escolheu a multiplicação. Digite o primeiro valor")
    n1 = int(input ())
    print ("Digite o segundo valor a ser multiplicado")
    n2 = int(input ())
    multiplicacao = n1*n2
    print ("o valor da multiplicação de ", n1, " vezes  ", n2, "  é ", multiplicacao)
elif (op==4):
    print ("Você escolheu a divisão. Digite o primeiro valor")
    n1 = int(input ())
    print ("Digite o segundo valor, que não pode ser zero")
    n2 = int(input ())
    if (n2==0):
        print ("não existe divisão por zero")
    else:
        divisao = n1/n2
        print ("o valor da soma de ", n1, " por  ", n2, "  é ", divisao)
else:
    print ("Opção inválida")
```